Contents

Processor Design Studio — Build Spec **1**
Objective . . . 1
Finalized Architecture . . . 1
Tech Stack (final) . . . 2
ISA (toy RISC, MIPS-style) . . . 2
Instruction Encoding . . . 2
Assembler Input Format . . . 3
Register & Memory Conventions . . . 3
Folder Structure . . . 3
Data Contracts . . . 4
Build Order (phase-gated — confirm acceptance criteria before advancing) . . . 5
Agent Instructions . . . 5

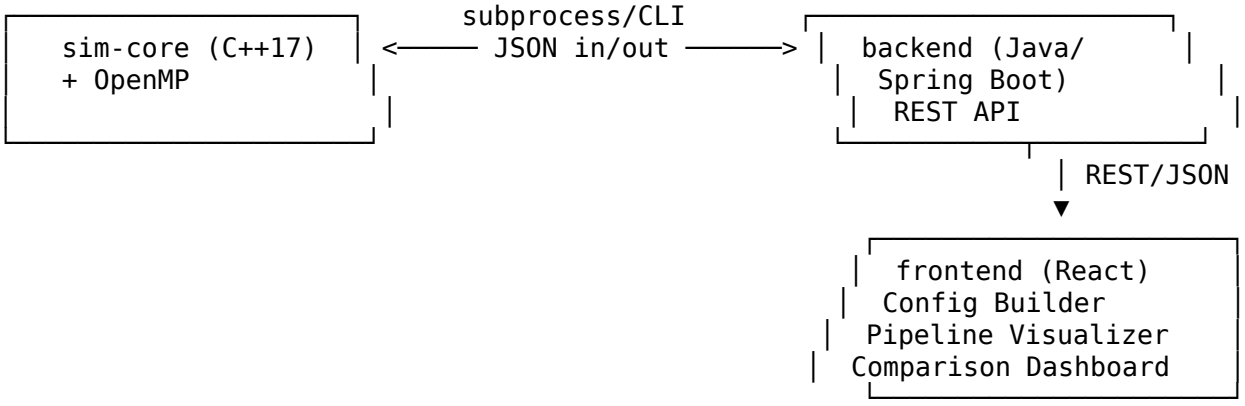
Processor Design Studio — Build Spec

> Paste this whole file into Antigravity as the master prompt. Work through the phases in order — do not skip ahead. Confirm each phase's acceptance criteria are met before starting the next. Ask before making architecture decisions not covered here.

Objective

Build **Processor Design Studio**: a platform that simulates a toy RISC processor (functional → pipelined → cache-equipped → branch-predicted), runs architecture-exploration sweeps across configurations in parallel with OpenMP, and visualizes/compares results in a web UI.

Finalized Architecture



Why this split: sim core needs OpenMP → must be C/C++. Backend stays Java (Spring Boot) since that's the stronger language here — it just shells out to the compiled sim binary via `ProcessBuilder`, passing a config JSON on stdin and reading a result JSON from stdout. Frontend is decoupled, talks only to the backend's REST API.

Tech Stack (final)

Layer	Choice
Sim core	C++17, OpenMP, CMake
Backend	Java 17, Spring Boot, Maven
Frontend	React + Vite, Recharts (charts), SVG/Canvas (pipeline animation)
Data interchange	JSON (schemas below)
Build	CMake (`-fopenmp`), Maven, npm

ISA (toy RISC, MIPS-style)

32-bit instructions, 16 general registers `$r0`–`$r15` (`$r0` hardwired to 0).

- **Arithmetic:** ADD, SUB, AND, OR, XOR, SLL, SRL, ADDI, ANDI, ORI
- **Memory:** LW, SW
- **Branch:** BEQ, BNE, BLT, BGE
- **Jump:** J, JAL, JR
- **Other:** NOP, HALT

(~19 instructions — enough to exercise hazards, loops, function calls, without full RV32I spec overhead.)

Instruction Encoding

All instructions are 32 bits, big-endian field layout. Three formats:

R-type (register-register: ADD, SUB, AND, OR, XOR, SLL, SRL, JR)

```
| opcode(6) | rs(4) | rt(4) | rd(4) | shamt(6) | funct(8) |
```

I-type (immediate/memory/branch: ADDI, ANDI, ORI, LW, SW, BEQ, BNE, BLT, BGE)

```
| opcode(6) | rs(4) | rt(4) | immediate(18, signed) |
```

J-type (J, JAL)

```
| opcode(6) | address(26, word index) |
```

NOP and HALT use opcode only, remaining bits zero.

Mnemonic	Format	Opcode	Funct (R-type only)
ADD	R	0x00	0x00
SUB	R	0x00	0x01
AND	R	0x00	0x02
OR	R	0x00	0x03
XOR	R	0x00	0x04
SLL	R	0x00	0x05
SRL	R	0x00	0x06
JR	R	0x00	0x07
ADDI	I	0x01	—
ANDI	I	0x02	—
ORI	I	0x03	—
LW	I	0x04	—

Mnemonic	Format	Opcode	Funct (R-type only)
SW	I	0x05	—
BEQ	I	0x06	—
BNE	I	0x07	—
BLT	I	0x08	—
BGE	I	0x09	—
J	J	0x0A	—
JAL	J	0x0B	—
NOP	—	0x3E	—
HALT	—	0x3F	—

Assembler Input Format

Plain-text `.asm` files, one instruction per line. Labels end in `:` on their own line or prefixing an instruction. Comments start with `#`. Registers are `$r0`–`$r15`.

```
main:
    ADDI $r1, $r0, 10      # r1 = counter = 10
    ADDI $r2, $r0, 0       # r2 = sum = 0
loop:
    BEQ  $r1, $r0, end     # if counter == 0, exit
    ADD  $r2, $r2, $r1     # sum += counter
    ADDI $r1, $r1, -1      # counter--
    J    loop
end:
    HALT
```

The assembler is a two-pass component of `sim-core`: pass 1 builds a label→address table, pass 2 emits the 32-bit encoded instruction stream per the table above. This is in scope for **Phase 1** — the functional simulator consumes assembled programs, not raw asm text.

Register & Memory Conventions

- `$r0` — hardwired to 0 (writes ignored)
- `$r15` — return address (RA); JAL writes PC+4 here automatically before jumping
- `$r14` — stack pointer (SP) by convention, for benchmarks that need one; not enforced by hardware
- `$r1`–`$r13` — general purpose, no other implicit role
- Memory — byte-addressable, word-aligned (4-byte) loads/stores only; default address space is 4KB (1024 words) for benchmark programs, defined as a constant in `sim-core` (bump it if a benchmark needs more)
- PC — starts at address 0, increments by 4 per fetch; branches (BEQ/BNE/BLT/BGE) are PC-relative (signed immediate = word offset added to PC after increment); J/JAL use an absolute word address (26-bit field × 4)

Folder Structure

```
processor-design-studio/
├── sim-core/
│   ├── src/
│   │   ├── isa/            (decoder, register file, memory)
```

```
│   │   ├── pipeline/       (stages, hazard detection, forwarding/stalling)
│   │   ├── cache/          (cache controller, replacement policies)
│   │   ├── predictor/      (branch predictors)
│   │   ├── sweep/          (OpenMP sweep runner, results aggregation)
│   │   └── main.cpp
│   ├── include/
│   ├── benchmarks/         (.asm test programs)
│   └── CMakeLists.txt
├── backend/
│   └── src/main/java/.../
│       ├── controller/     (ConfigController, SweepController)
│       ├── service/        (SimExecutorService — ProcessBuilder wrapper)
│       └── model/          (Config, Result DTOs)
├── frontend/
│   └── src/
│      ├── pages/         (ConfigBuilder, SweepRunner, PipelineView, Dashboard)
│       └── components/
└── docs/
```

Data Contracts

Config (input to sim-core):

```
{
  "benchmark": "matrix_multiply",
  "pipeline": { "enabled": true, "forwarding": true, "stalling": true },
  "cache": { "sizeKB": 32, "blockSize": 64, "associativity": 4, "replacement": "LRU", "wri
  "branchPredictor": { "type": "TWO_BIT_SATURATING" }
}
```

Result (output from sim-core):

```
{
  "config": { "...": "echoed back" },
  "totalCycles": 12345,
  "instructionsExecuted": 5000,
  "cpi": 2.47,
  "ipc": 0.405,
  "cacheHitRate": 0.87,
  "branchMispredictionRate": 0.12,
  "stallCycles": 890
}
```

Sweep result (array of Results + parallel metadata):

```
{
  "threadCount": 4,
  "sweepWallTimeMs": 812,
  "speedup": 3.1,
  "efficiency": 0.775,
  "results": [ /* array of Result objects, one per config */ ]
}
```

Build Order (phase-gated — confirm acceptance criteria before advancing)

Phase 1 — Assembler + functional simulator Two-pass assembler (label resolution → encoding, per the Instruction Encoding / Assembler Input Format above) feeding a fetch-decode-execute loop. Register file, flat memory array, instruction decoder. *Done when:* hand-written asm (factorial, loop sum, bubble sort) assembles correctly and produces correct register/memory state on execution.

Phase 2 — Pipeline + hazards 5-stage (IF/ID/EX/MEM/WB). Structural/data/control hazard detection. Forwarding and stalling as independently toggleable flags in config. *Done when:* CPI changes measurably and correctly when forwarding/stalling toggled off vs on, on a hazard-heavy benchmark.

Phase 3 — Cache hierarchy Configurable size/block size/associativity/replacement policy (LRU/FIFO/Random), write-through no-allocate. *Done when:* hit rate varies correctly across associativity/size sweeps on a memory-heavy benchmark.

Phase 4 — Branch prediction STATIC_NOT_TAKEN, STATIC_TAKEN, ONE_BIT, TWO_BIT_SATURATING. Track mispredictions + pipeline flush penalty. *Done when:* misprediction rate drops as predictor sophistication increases on a branch-heavy benchmark.

Phase 5a — Sequential sweep runner Given an array of configs, run each sequentially, collect Result JSON per config. *Done when:* full sweep over a config grid (cache size × associativity × predictor type) completes and matches individually-run results.

Phase 5b — OpenMP parallelization `#pragma omp parallel for` over the config array (each thread runs one full simulation, independent state per thread — no shared mutable sim state). Measure wall time at 1/2/4/8 threads via `omp_get_wtime()`. Compute speedup and efficiency. *Done when:* results are identical to the sequential run (correctness first), and speedup/efficiency numbers are captured per thread count.

Phase 6 — Benchmark suite 4–6 asm programs: tight loop (branch-heavy), array sum (memory-heavy), recursive fib (hazard-heavy), matrix multiply (mixed).

Phase 7 — Backend Spring Boot REST API: P0ST `/sweep` (accepts config array, invokes sim-core binary via ProcessBuilder, returns sweep result), GET `/benchmarks` (lists available programs).

Phase 8 — Frontend - Config Builder: form to define a config or a sweep grid - Pipeline Visualizer: animate instructions through IF/ID/EX/MEM/WB, color-coded stalls - Comparison Dashboard: charts (CPI vs cache size, misprediction vs predictor type, speedup vs thread count via Recharts)

Phase 9 — Report + docs Auto-generate a results write-up (charts + table) per sweep run. README with architecture diagram and sample results.

Agent Instructions

1. Work phase by phase in the order above. Do not start Phase N+1 until Phase N's "Done when" criterion is demonstrably met.
2. Sim core: use CMake, compile with `-O2 -fopenmp -Wall`. Keep per-thread sim state fully isolated (no shared mutable globals across OpenMP threads) — this is a correctness requirement, not a style preference.
3. Backend: no business logic beyond invoking the binary and shaping JSON — keep it a thin wrapper.

4. Write unit tests for the ISA decoder and hazard detection logic before moving to cache/predictor phases.
5. After Phase 5b, produce a short benchmark log: wall time and speedup/efficiency at 1/2/4/8 threads, plus one sentence identifying the serial fraction (config load, result aggregation) limiting max speedup — needed for the Amdahl's law discussion in the final report.